Step 10 Summary — Population-Based Training and Evolutionary Game Theory

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

10 Step 10 — Population-Based Training and Evolutionary Game Theory

This is a ground-up chapter on training and evaluating **populations** of agents: the evolutionary mathematics that says whether a population can settle at all, the transitive/cyclic structure that decides whether self-training converges or spins, and a small AlphaStar-style **league** built on a solvable poker game. It serves two purposes — a self-contained refresher for later steps and a primary source for the thesis's population-level contributions. It is written to be read on its own. **All experimental numbers reported here were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (analytic ESS/Nash for the matrix games; Step 07's exact best-response exploitability for Leduc). Where a run contradicted what theory led me to expect, I keep the original expectation and reconcile it with what happened — those gaps are the most instructive parts of the step.

Where this sits in the thesis. Steps 2-8 solved *one* fixed game; Step 9 added a *second* learner. Step 10 goes to a *whole population* that trains against itself. It carries three thesis hooks. **Main exploiters** are automated opponent modelers — the population lift of Step 7's static read (Contribution #1). The **AlphaStar league** is a population-level safe-exploitation mechanism, but one whose guarantee is *missing* — the exact place Contribution #2 lives. And **EGTA / meta-Nash** is the population evaluation

methodology (Contribution #3).

10.1 1. Why populations — and why structure decides everything

Self-play has a famous failure mode: a player that only ever trains against its current self can go around in circles. If the game is **rock-paper-scissors**, "get better" has no meaning — Rock beats Scissors, loses to Paper, and improving against one opponent just makes you worse against another. The single most important idea in this step is that games have two kinds of structure mixed together: a **transitive** part (a genuine skill ladder — there *is* a better player) and a **cyclic** part (a wheel of counters — there is only *what beats what*). Whether a self-training population converges to something good or spins forever is decided by *which part dominates*, not by the learning rate.

A picture to hold onto: a **dojo**. The **main** students are what you are trying to produce. You keep a room full of **sparring partners** whose entire job is to find and punish a specific weakness in the current students (exploiters). And you keep a **museum of past champions** — frozen snapshots — so nobody wins by forgetting how to beat an old style. Training is then a loop: students spar, the strong ones get copied and slightly mutated (population-based training), and periodically a copy of everyone is frozen into the museum. The league is that dojo, made mathematical.

> **Read more:** Balduzzi, D. et al. (2019). "Open-ended Learning in Symmetric Zero-sum Games." *ICML* — the spinning-top geometry of transitive vs cyclic structure; and Jaderberg, M. et al. (2017). "Population Based Training of Neural Networks." *arXiv:1711.09846*.

10.2 2. The family of methods

Every method here is a different way to turn "a population" into a training signal or an evaluation.

Approach	What it does	Reach for it when	Main weakness
Self-play	train the latest policy against (a copy of) itself	transitive games; a quick baseline	cycles / forgets in cyclic games; last iterate does not converge
PSRO	population + meta-Nash + best-response oracle; add the BR, repeat	competitive games; want a game-theoretic target	a full best response per round; the *mixture* is not the answer (see §7)
PBT	train many agents; copy the fitter ones (exploit) + perturb hyper-parameters (explore)	any population; online hyper-parameter search	can collapse diversity or regress (see §5)

Approach	What it does	Reach for it when	Main weakness
AlphaStar league	PBT with three agent *types* (main / main-exploiter / league-exploiter) + freezing + PFSP	non-transitive games where naive self-play cycles	many agents, much compute; heuristic, no safety guarantee

Two evaluation tools cut across them. **Replicator dynamics** (§3) are the continuous idealization of selection — they say where a population *flows* and where it can *rest*. The **spinning-top decomposition** (§4) measures how transitive vs cyclic a game (or a population) is. Together they are the diagnostic that predicts, *before* you spend the compute, whether the league in §5 will settle or spin.

Historically the line runs: population-based training as online hyper-search (PBT, 2017) → game theory over a policy population (PSRO, 2017) → the geometry that explains *why* you need a population (spinning-top, 2019) → and the full three-type league that beat human pros (AlphaStar, 2019), evaluated with empirical game theory (EGTA, Tuyls et al., 2020).

10.3 3. Replicator dynamics — where a population can rest

The **replicator equation** is the simplest model of selection: a strategy's share grows in proportion to how much it beats the *population average*,

$$\dot{x}_i = x_i \left[f_i(x) - \bar{f}(x) \right], \qquad \bar{f}(x) = \sum_j x_j f_j(x).$$

Its rest points are exactly Nash equilibria; the *stable* rest points are **evolutionarily stable strategies (ESS)** — mixes that, once adopted by the whole population, cannot be invaded by a small mutant share. This is the continuous-time idealization of what a PBT league does discretely: copy the fitter agents (selection) and perturb them (mutation).

Run on four canonical symmetric games (deterministic dynamics, so the outcome is unambiguous), the measured results match theory exactly:

Game	Analytic reference	Measured final	Converged?
Prisoner's Dilemma	Defect dominates; ESS $(0, 1)$	$[0.0, 1.0]$	yes
Hawk-Dove	interior ESS $p(\text{Hawk}) = V/C = 0.5$	$[0.5, 0.5]$ (orbit radius 0.0)	yes

Game	Analytic reference	Measured final	Converged?
Rock-Paper-Scissors	Nash = uniform; **no ESS**	orbits centre, radius 0.095	**no**
Stag Hunt	two pure ESS (basin-dependent)	$[0.8, 0.2] \to [1, 0]$; $[0.2, 0.8] \to [0, 1]$	yes

Prisoner's Dilemma collapses to the dominant strategy; Hawk-Dove *reaches* the interior 0.5 ESS; Stag Hunt picks a different pure ESS depending on the basin its start lands in. **Rock-Paper-Scissors never converges** — its interior fixed point is a *centre*, so the population orbits it forever. That last row is the whole motivation for the rest of the step: a cyclic game has no stable population.

> **Read more:** Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) — replicator dynamics, ESS, and the RPS centre; and the Bloembergen–Tuyls survey (JAIR, 2015) connecting replicator dynamics to multi-agent learning.

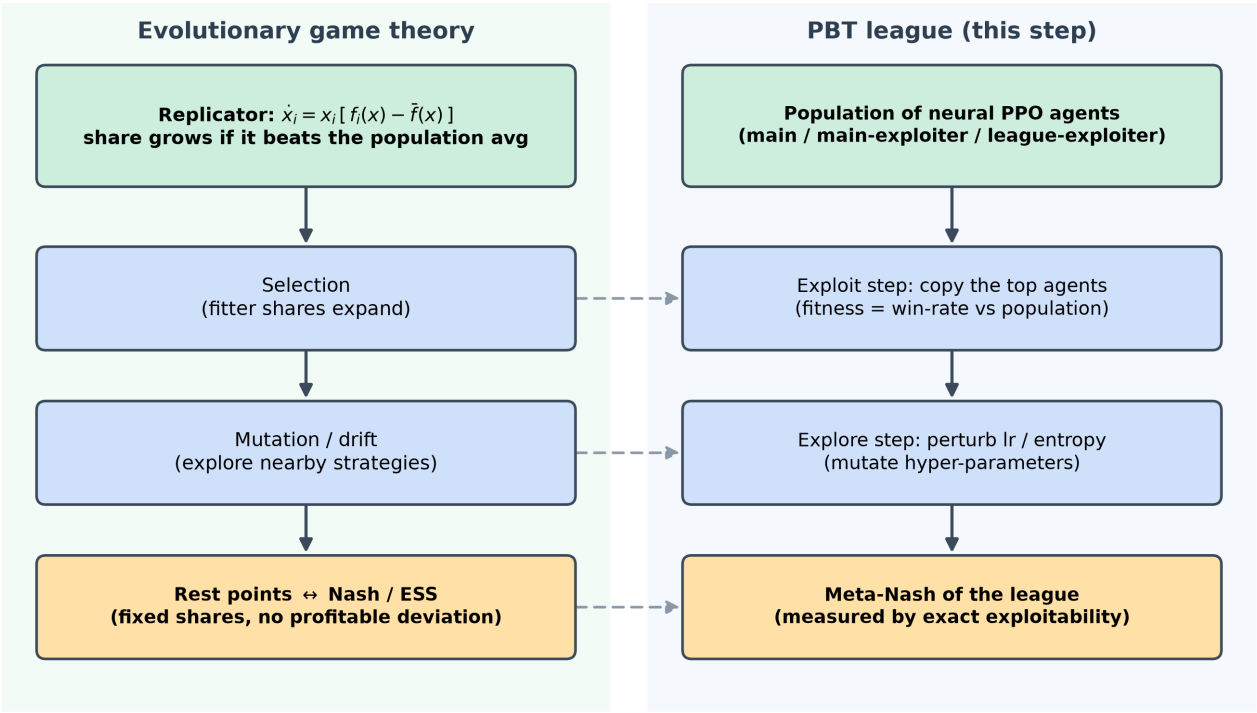


Figure 1: Evolutionary game theory as the continuous idealization of a PBT league: replicator selection ↔ copying the fitter agents, mutation/drift ↔ perturbing learning rate and entropy, and replicator rest points ↔ the meta-Nash / ESS the league is trying to reach. The league is replicator dynamics with learned agents instead of fixed strategy shares.

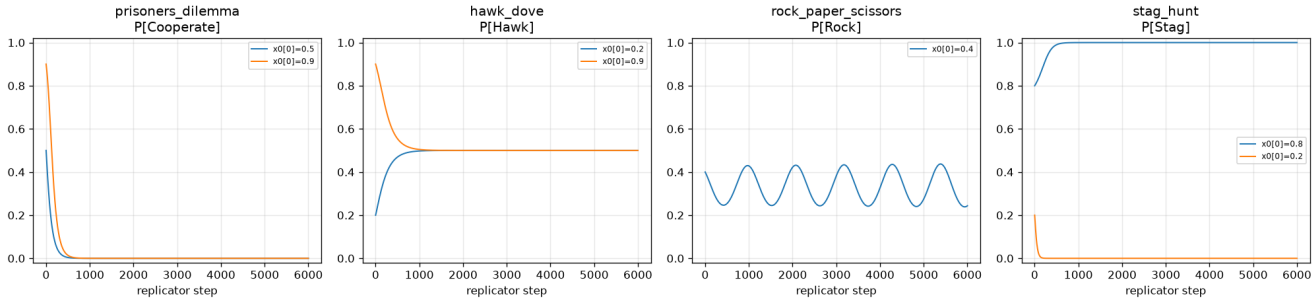
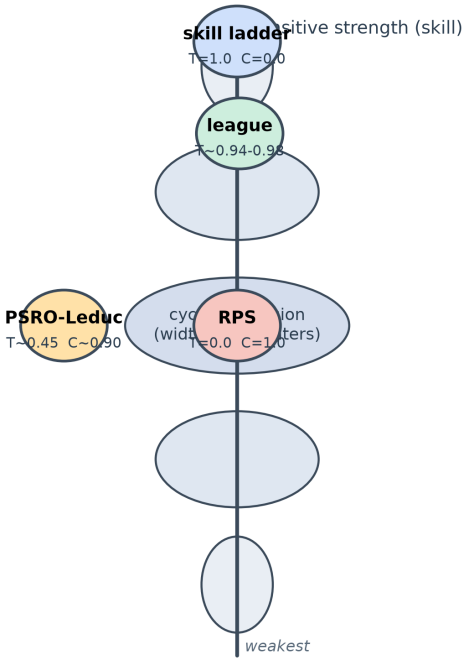


Figure 2: Replicator phase portraits: Prisoner's Dilemma → all-Defect, Hawk-Dove → the 0.5 interior ESS, Rock-Paper-Scissors → a closed orbit that never converges, and Stag Hunt → two basins (all-Stag or all-Hare). The non-converging RPS orbit is why populations in cyclic games need explicit machinery to avoid spinning.

10.4 4. The spinning top — transitive vs cyclic structure

If cyclic games are the problem, we need to *measure* how cyclic a game is. The **spinning-top decomposition** (Balduzzi et al.) splits a game's payoff matrix into a **transitive** component (a skill ladder, captured by per-strategy ratings) and a **cyclic** component (what is left over — the rock-paper-scissors part). The name comes from the shape: real games are widest (most cyclic) among mediocre strategies and narrow to a point (purely transitive) at the extremes of skill.



same game, different populations: a best-response population (PSRO) sits in the wide cyclic belly; a training-snapshot population (league) climbs the transitive spine.

Figure 3: The spinning top: a vertical transitive axis (skill — there is a better player) and a cyclic dimension (width — the number of counters, rock-paper-scissors structure). Rock-Paper-Scissors sits in the widest cyclic belly (transitive ratio 0.0); a pure skill ladder sits on the transitive spine (1.0); the PSRO-Leduc best-response meta-game sits in the wide cyclic belly (~0.45 transitive), while the league's snapshot meta-game climbs the transitive spine (~0.94-0.98).

A subtlety worth flagging: the raw step suggested an **SVD rank-1** decomposition, which wrongly reports Rock-Paper-Scissors as ≈ 0.707 transitive — a rank-1 skew-symmetric approximation simply cannot represent a pure cycle. The implementation uses the **combinatorial-Hodge** (ratings-difference) method instead, which correctly gives RPS a transitive ratio of 0.0. Measured:

Population	Transitive (Hodge)	Cyclic	Structure
Rock-Paper-Scissors	0.0	1.0	purely cyclic
Pure skill ladder	1.0	0.0	purely transitive

Population	Transitive (Hodge)	Cyclic	Structure
PSRO-Leduc **best-response** meta-game	0.41-0.46	0.89-0.91	**mostly cyclic** (27 three-cycles)
League **snapshot** meta-game	0.94-0.98	—	**mostly transitive**

The two *real* populations are the headline — and they disagree, on the same game.

> **Reconciliation (kept prediction → what actually happened).** I framed poker as a skill ladder and expected Leduc's meta-game to be mostly transitive. Measured, it depends entirely on *which population you decompose*. A population of **best responses** (PSRO) is mostly **cyclic** (≈ 0.45 transitive, 27 three-cycles): the best response beats the current mixture, a newer best response beats *that*, and so on — Balduzzi's spinning-top in action. A population of **training-trajectory snapshots** (the league) is mostly **transitive** (≈ 0.94-0.98), because later snapshots are usually stronger than earlier ones, forming a ladder. Neither is a bug; the transitive/cyclic ratio is a property of the *population*, and choosing how you build the population is choosing whether you see a wheel or a ladder.
>
> **Read more:** Balduzzi, D. et al. (2019). "Open-ended Learning in Symmetric Zero-sum Games." *ICML*; and Czarnecki, W. M. et al. (2020). "Real World Games Look Like Spinning Tops." *NeurIPS*.

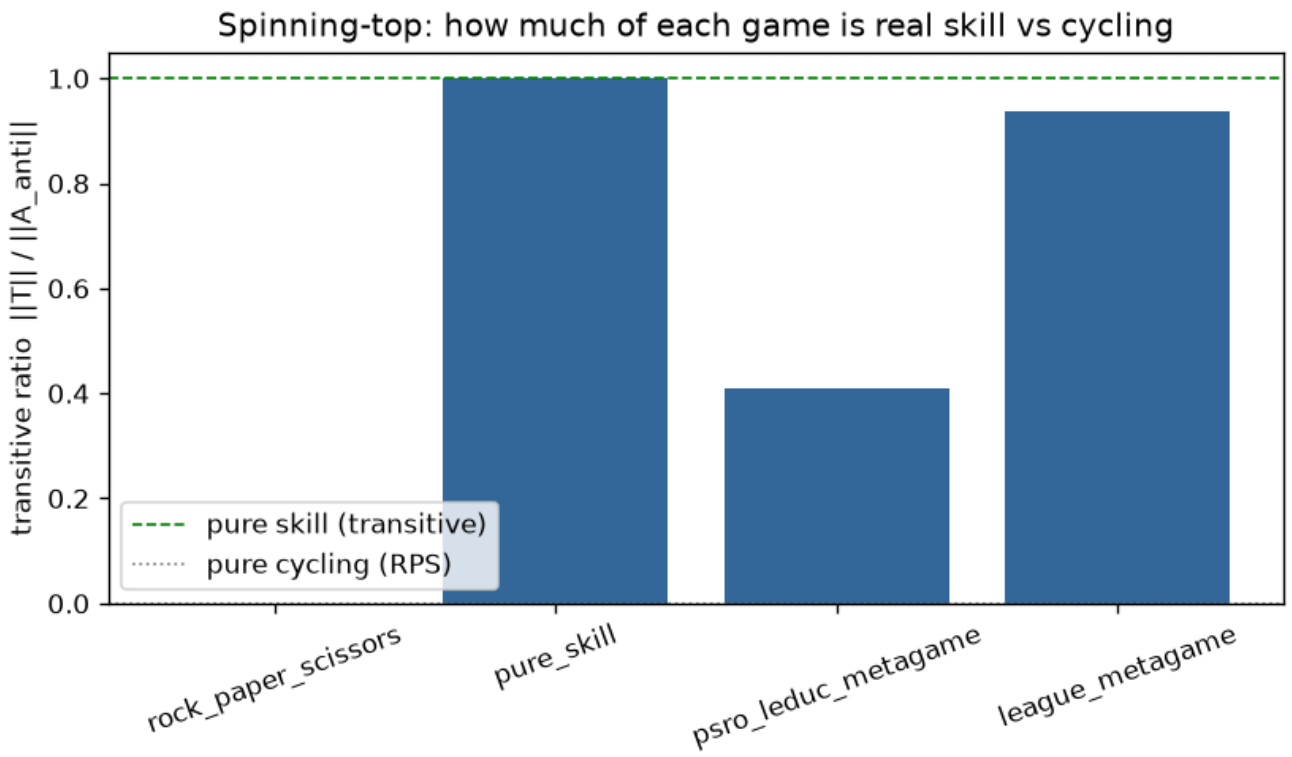


Figure 4: Transitive ratio across four populations: RPS (0.0, purely cyclic), a pure skill ladder (1.0), the PSRO-Leduc best-response meta-game (~0.41-0.46, mostly cyclic), and the league snapshot meta-game (~0.94-0.98, mostly transitive). Same game, opposite structure, depending on how the population is built.

10.5 5. The AlphaStar-style PBT league

With the diagnostic in hand, we build the dojo. The league trains neural PPO agents on **Leduc Hold'em** with three agent types: **main** agents (the product; they play everyone via PFSP self-play), **main exploiters** (hunt weaknesses in the *current* mains), and **league exploiters** (hunt weaknesses anywhere in the *frozen history*). Population-based training copies the top agents (exploit) and perturbs their learning rate / entropy (explore); periodically a frozen snapshot of each agent is added to the museum; **PFSP** matchmaking samples each opponent with probability rising in how hard it is to beat. Critically, every neural network is extracted to a **tabular** policy so Step 07's **exact** best response measures its exploitability — the same NashConv used since Step 2.

Does it improve? Measured, over two configs (smoke: 7 agents, 15 epochs; scale: 8 agents, 120 epochs, 48 frozen snapshots):

Metric	Smoke	Scale
min-main exploitability	4.67 → 3.04 (ends at min)	4.73 → **min** ≈ 1.21 → 2.05
meta-Nash exploitability	4.73 → 3.04	5.01 → min ≈ 1.32 (plateau 1.60) → 2.96
final Elo (live agents)	1176-1211	1198-1210

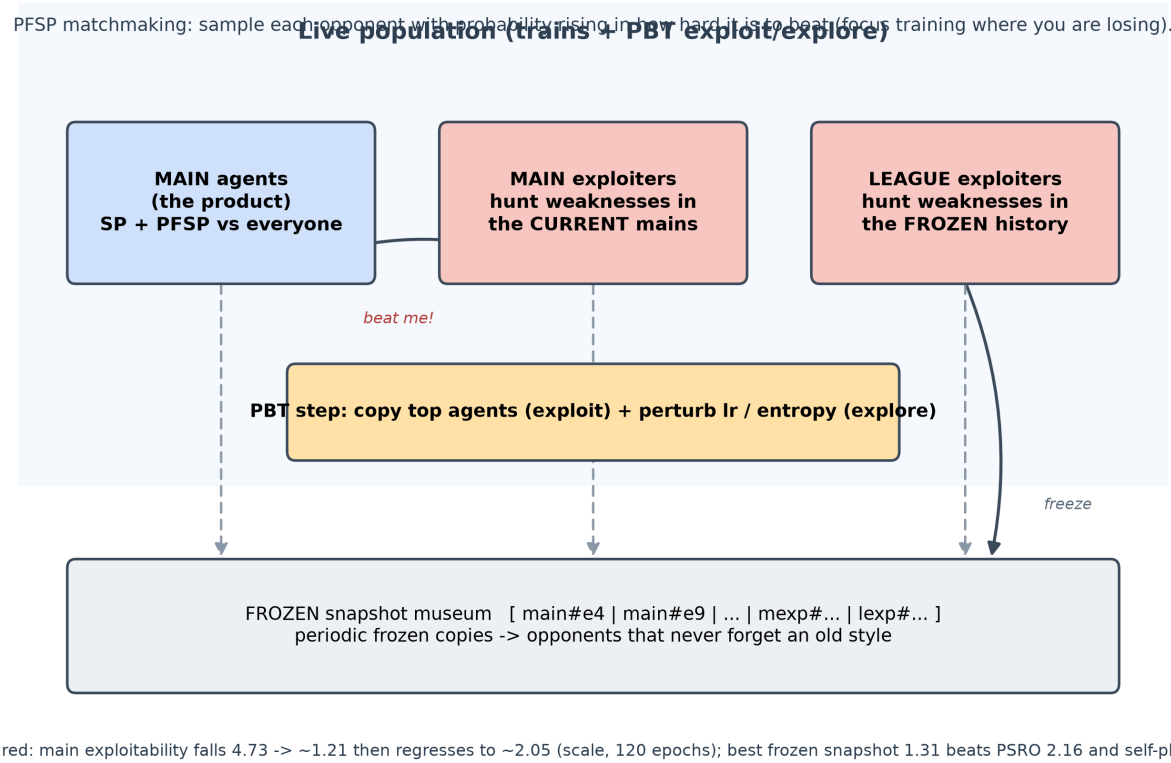


Figure 5: The league: main agents (the product), main exploiters (hunt weaknesses in the current mains), and league exploiters (hunt weaknesses in the frozen history), with periodic freezing into a snapshot museum and PFSP matchmaking that focuses training where the agent is losing. PBT copies the top agents and perturbs their hyper-parameters.

Reconciliation (kept prediction → what actually happened). I predicted a *monotone* decrease in exploitability. Smoke's 15 epochs oblige — a clean drop that ends at its minimum. But scale's 120 epochs tell the real story: exploitability falls steeply to a minimum near epoch 60 (min-main $\approx$ 1.21, meta-Nash bottoming $\approx$ 1.32 then holding a $\approx$ 1.60 plateau), and then **regresses** back up to $\approx$ 2.05 / $\approx$ 2.96 by epoch 119. The best agents are the *frozen snapshots* from mid-run; the *live* main agents get worse late, chasing their exploiters (churn / partial forgetting). This is only visible once training is long enough — a running league is not a monotonically improving one. The remedy (untested here) is best-snapshot retention / population regularization. Methodologically it echoes Step 9: **scale reveals what smoke hides.**

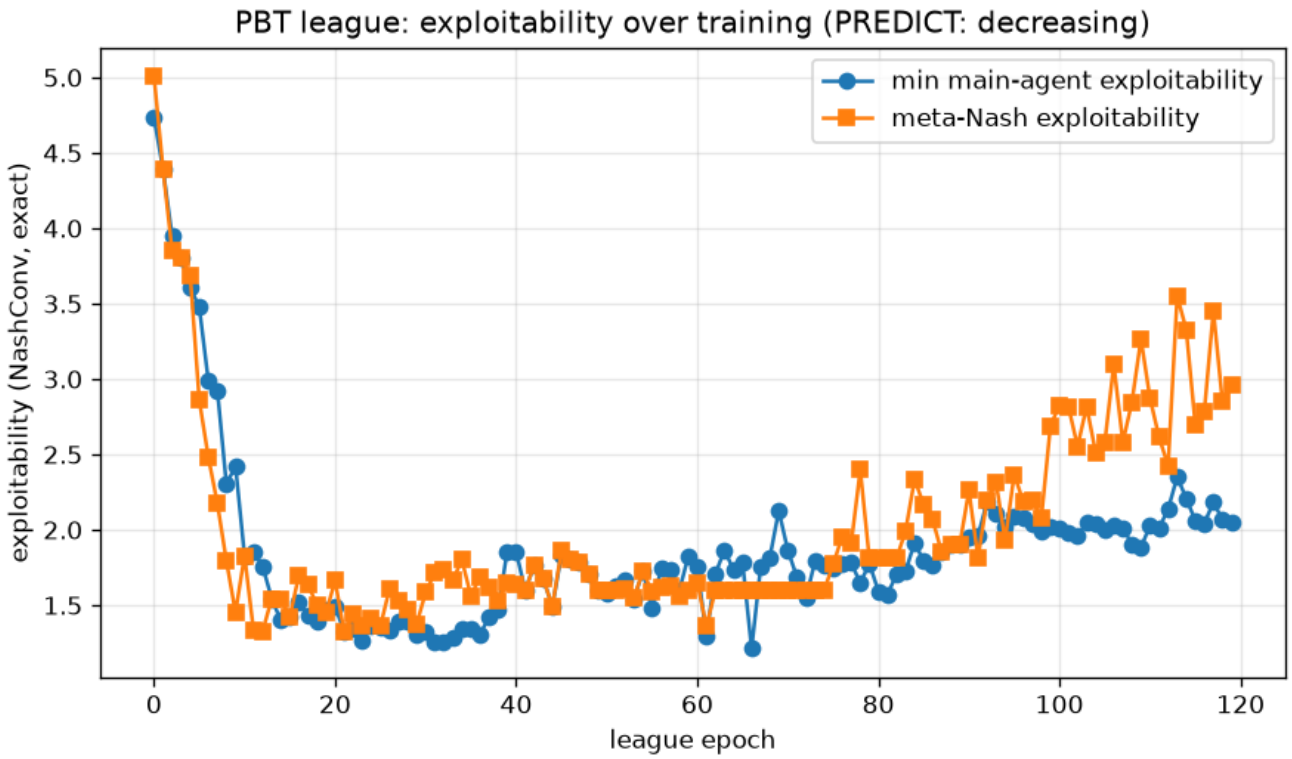


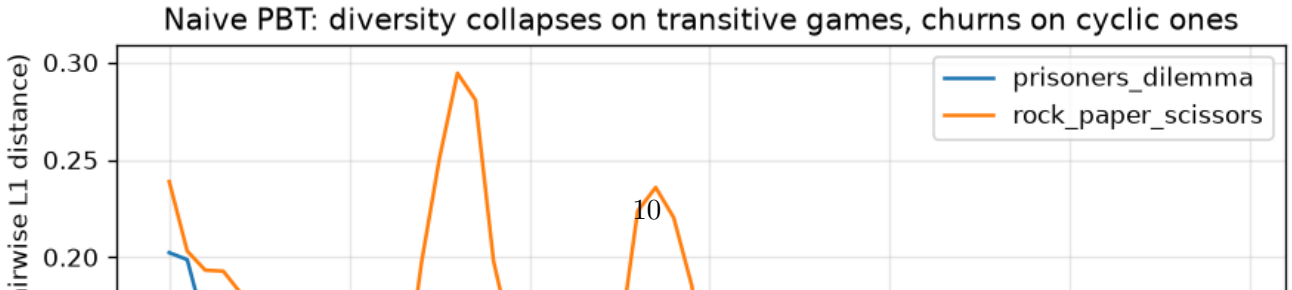
Figure 6: League exploitability over 120 epochs (scale): both min-main and meta-Nash exploitability fall to a minimum near epoch 60 and then regress upward. The frozen snapshots capture the strong mid-run agents; the live agents degrade late under exploiter pressure.

10.6 6. Diversity — is the population actually diverse?

A league is only as good as the *variety* of strategies it holds. We measure three things: the **effective population size** (participation ratio of the meta-Nash weights), **behavioral clustering** (are the policies actually different?), and **exploit coverage**. Measured, the population is only weakly diverse: participation ratio rises from 1.0 (smoke) to 1.9 (scale), but behavioral clustering collapses everything into a **single cluster** at both scales — even at scale, where the maximum pairwise behavioral distance (0.48) exceeds the clustering threshold (0.30), single-linkage merges the chain. Diversity here is *weight-level* (the meta-Nash spreads support over a few agents), not *behavior-level* (the agents play near-identically).

The mechanism is clearest on a fast toy — a mini-PBT on matrix games:

Game	Diversity over generations
Prisoner's Dilemma (transitive)	collapses to 0 (everyone → Defect)
Rock-Paper-Scissors (cyclic)	churns forever (0.07-0.29), never settles



not materialize at this scale.

> **Read more:** Vinyals, O. et al. (2019). "Grandmaster level in StarCraft II using multi-agent reinforcement learning." *Nature* (AlphaStar; the league and PFSP).

10.7 7. EGTA — evaluating the population, and a surprise

The last tool is **empirical game-theoretic analysis (EGTA)**: treat whole policies as the "strategies" of a higher game, play every pair to fill an empirical payoff matrix, solve its **meta-Nash** mixture, and score it. It is the population lift of exploitability — Nash of a game whose atoms are policies.

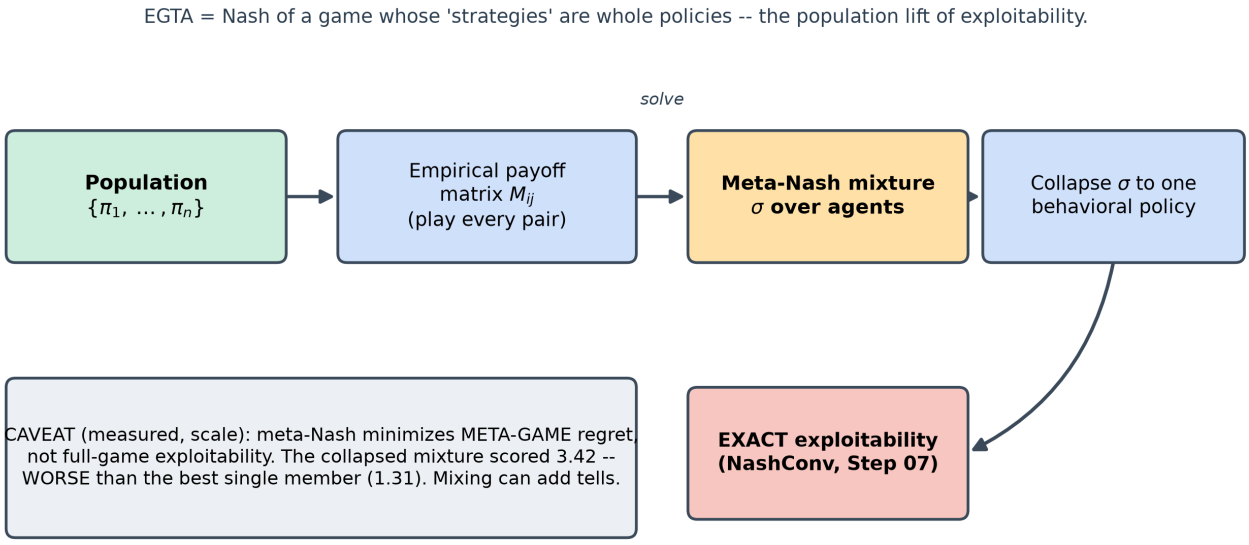


Figure 8: The EGTA pipeline: play every pair of agents to build an empirical payoff matrix, solve its meta-Nash mixture, collapse the mixture to a single behavioral policy, and measure its EXACT full-game exploitability. The measured caveat: the meta-Nash minimizes meta-game regret, not full-game exploitability, so the mixture can be more exploitable than the best single member.

The expectation (from the raw step's checklist) was that the meta-Nash of the league would be *less* exploitable than any single member. Measured:

Config	meta-Nash exploitability	best individual	meta-Nash ≤ best?
Smoke	2.665	2.665	**yes** (all weight on the best agent)
Scale	3.418	1.305	**no**

Reconciliation (kept prediction → what actually happened). I expected the meta-

Nash mixture to be at least as unexploitable as its best member. Smoke confirmed it trivially — the meta-Nash put *all* weight on the single best agent, so meta = best = 2.665. At scale the meta-Nash spreads weight (0.645 on one agent plus a tail), and the collapsed behavioral mixture scores 3.418 — **worse** than the best single snapshot (1.305). This is not a mixing bug (the identical code path gave meta = best in smoke). The meta-Nash minimizes **meta-game regret** — doing well *against the population* — which is a *different objective* from minimizing **full-game exploitability**; and a realization-weighted mixture of behavioral policies can be *more* exploitable than its components, because the blend introduces information-set "tells" that an exact best responder punishes. The lesson inverts: what a league should *ship* is a selected, best-response-robust member — not the meta-Nash mixture.

How does the league compare to the alternatives on Leduc exploitability?

Method (scale)	Final exploitability
CFR-Nash (the floor)	0.0099
League — best individual	1.305
PSRO (exact best-response oracle)	2.163
Self-play	3.683
League — meta-Nash mixture	3.418

The league's **best individual** is the strongest learned result — beating exact PSRO and self-play — though all learned methods remain far above the CFR-Nash floor. The league's **mixture** is the weakest, worse even than self-play (the §7 reconciliation). Ship the member, not the mixture.

Read more: Lanctot, M. et al. (2017). "A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning." *NeurIPS* (PSRO); and Tuyls, K. et al. (2020). "Bounds and dynamics for empirical game-theoretic analysis." *AAMAS/JAAMAS* (EGTA).

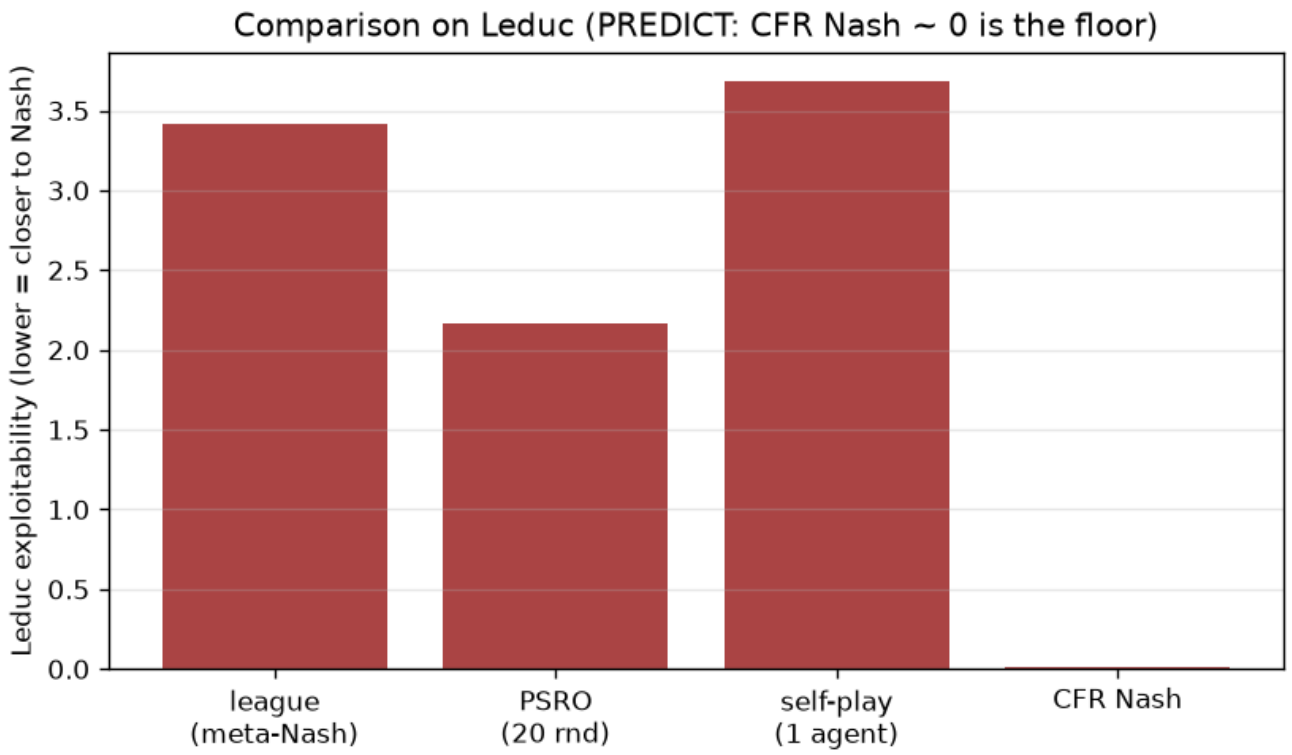


Figure 9: Final Leduc exploitability by method (scale): CFR-Nash floor ~0.01; the league's best individual (1.31) beats PSRO (2.16) and self-play (3.68); the league's meta-Nash mixture (3.42) is the weakest learned result.

10.8 8. Honest notes, limitations, and where this hands off

What held up. The confirmed backbone: replicator dynamics reproduce every analytic ESS/Nash, including RPS's non-converging orbit; the Hodge spinning-top decomposition cleanly separates skill from cycles and correctly labels the pure cases; the league drives strong *early* improvement and produces a best individual (1.305) that beats PSRO (2.163) and self-play (3.683); and EGTA gives a working population-level exploitability. Together they trace the arc from "which games can a population even settle in?" to "what happens when a population trains itself?"

What did not, and why it matters. Three honest caveats travel forward. (1) The league's exploitability **regresses late** at scale ($4.73 \to\approx 1.21 \to\approx 2.05$) — a running league is not a monotonically improving one; the best agents are frozen snapshots. (2) The **meta-Nash mixture is more exploitable than its best member** at scale (3.42 vs 1.31) — mixing behavioral policies does not guarantee robustness, so ship a member, not the mixture. (3) Leduc's meta-game is **cyclic as a best-response population** but **transitive as a snapshot population** — structure is a property of the population you build. And a methodological point: the late regression and the mixing failure are both invisible at the fast smoke config — scale reveals what smoke hides.

Trust. Every evolutionary target is *analytic* (ESS/Nash of the matrix games), and every league exploitability is Step 07's *exact* NashConv on tabular-extracted policies — so the game-theoretic results

are bounded by ground truth. The neural results are a single PBT run per config: the *directions* (early improvement, best-individual < PSRO < self-play, late regression, meta-Nash > best member) are the trustworthy claims, not the third-decimal magnitudes. The experiment PNGs above are produced from the committed JSON artifacts (see `../figures/README.md`); the conceptual diagrams are generated by the `make_*_figure.py` scripts.

Backward and forward connections. Backward: the league is Step 9's PSRO made asynchronous with neural oracles, and it reuses Step 07's exact best response wholesale; the "meta-Nash of a population" is Step 2's Nash lifted one level. Forward: main exploiters are automated opponent modelers (Contribution #1); EGTA/meta-Nash is the evaluation methodology (Contribution #3); and the league's missing guarantee — it can regress and its mixture can be exploitable — is the population form of the **missing $N > 2$ safety anchor** (Contribution #2). The transitive/cyclic diagnostic predicts Step 11's FFA coalition games will be strongly cyclic, so naive PBT there will cycle.

Open questions. Does best-snapshot retention / population regularization remove the late regression (a training fix), or is it inherent to the three-type design (a design fix)? Should population evaluation report a selected member or a diversity-regularized meta-solver instead of the meta-Nash mixture? And underneath both: with no minimax value for a population, what does "safe" mean, and can the league's exploiter mechanism be given a guarantee rather than remaining heuristic?

10.9 Key takeaways for the thesis synthesis

- **Structure decides convergence.** The transitive/cyclic (spinning-top) ratio is a *pre-training diagnostic*: transitive games settle (and kill diversity), cyclic games spin (and force diversity). Measured: RPS 0.0 transitive, skill ladder 1.0, PSRO-Leduc ≈ 0.45 (cyclic), league snapshots ≈ 0.94-0.98 (transitive).
- **The league produces strong individuals but carries no guarantee.** Best snapshot 1.305 beats PSRO 2.163 and self-play 3.683 — yet exploitability *regresses* late ($\rightarrow 2.05$) and the meta-Nash *mixture* (3.42) is worse than its best member (1.31).
- **Meta-Nash optimizes meta-game regret, not full-game exploitability** — the two disagree, so population evaluation must score the collapsed mixture directly and ship a selected member.
- **Exact evaluation is the anchor.** Extracting neural policies to tabular and grading them with Step 07's exact best response makes every population claim ground-truthed — the same NashConv since Step
  2.
- **The population-safety gap is the open door (Contribution #2):** the AlphaStar exploiter mechanism is the population analog of Step 8's safe exploitation, but it is heuristic — it neither guarantees monotone improvement nor a non-exploitable mixture.